

Technical Assessment Questions

Thank you for working through the Patronus Technical Assessment. Below you will find 5 problem statements. You can choose which questions you would like to solve, but only need to complete one problem. You can attempt others or provide descriptions of how you might solve them, but it is not necessary. We expect you to spend 1.5 to 2 hours on this assignment.

The goal here is to give you a few opportunities to demonstrate your knowledge in a domain that is important to us. While it is written in code, how something works is secondary to a demonstration of knowledge in the domain. In other words, I'm more interested in your explanations of what you do and your defense of your decisions rather than getting 100% bug free code. At the end of each statement you will see a description of deliverables and the evaluation criteria.

Problem 1: Faster than Numpy?

Numpy is a ubiquitous highly efficient numerical methods package for python. We use it regularly for matrix math/linear algebra problems as well as for probability and optimization.

For this question, write a function that completes a computation more quickly than the equivalent numpy calculation. This should be in the matrix math/linear algebra domain.

While there are some languages that have an advantage over python in this domain, please write the function in python. To demonstrate the speed of the functions please use the cProfile or timeit packages. The deliverable should be two functions and the code you used to assess them. Our preference would be a jupyter notebook. You may also want to include 1-2 slides or a 1 page doc with commentary.

Evaluation

Ideally, you will be able to identify a use case where your non-numpy function actually performs faster. However, if you find one that is very close to the numpy implementation or one that is not significantly faster, that is okay as well. The evaluation will be around your understanding of numpy's strengths and weaknesses and how it can be used optimally. We will talk about vectorization and the other elements of data structures, memory usage, typing, and design that determines the performance of these type of computations.

Problem 2: Vibe Coding

Generative AI via LLM can be great for developer productivity. At the most simple end, code generation platforms can automate boilerplate and generate functions that are commonly re-used. Generative AI can explore API documentation and provide useful code and can help people understand how to use new packages and methods.

There are limitations. Since LLM's don't reason and can sometimes hallucinate, they can produce code that looks good, but doesn't actually work. For this exercise, I would like you to do the following.

1. Give the generative AI tool of your choice a prompt to generate code.
2. See if you can get the tool to generate in-operative or in some way faulty code.
3. Critique the code and correct the LLM's error

It's harder than you think to get and LLM to write truly faulty code. You may need to iterate a bit. The deliverable should be 3-4 slides (or two pages in a doc) that outline the prompt you input, the code you received and the evaluation of that code.

Evaluation

Why did the LLM produce faulty code? Would like to understand your assessment of generative AI based code. How you use it and what you do to use it safely. While there isn't a "right" answer here, better responses will demonstrate strong understanding of commodity code and the differences between spec code and production code.

Problem 3: Micro-microservice

This is a continuation of the screening interview problem. I would like you to build out a micro-microservice. Please select a dataset from the Kaggle library of open datasets.

[Kaggle Data](#)

You will build both the server and the consumer of the data. While you can build any collection of endpoints you like, there needs to be one to get data from the source dataset with at least one input parameter and one to post data to the dataset.

Unlike in the screening interview, this time you have a bit of time to focus and I would like you to implement a more robust data pipeline. Please focus on data validation and error handling. Try to design something that considers future needs, it should be extensible and scalable. This can be written in any language. The deliverable is the data and code. For ease of evaluation, docker may be the best way to deliver this project. However, since the evaluation only requires that the server be accessible to me and I can see your code examples, there are a variety of ways to deliver.

Evaluation

I will use your code (or write my own) to get and put data from the server. I will push the server code a bit to see how it handles more or faster requests. While the code should be generally operable, it is okay if it has bugs. We will spend most of our time discussing design decisions. Be prepared to talk about how you might enhance the system.

Problem 4: Build a better beta

Much of the Patronus ecosystem is built upon open source python libraries. Because we are a small team we don't have the resources to build out all the functionality we need. Instead we leverage the work of others. Picking and choosing the libraries and platforms we use is an important part of an engineer's role.

For this problem, we would like an assessment of the packages we use at Patronus. What are the strengths and weaknesses of those packages, are there ones you would recommend instead.

Apologies as this is a bit of "free work" we are asking for here. The deliverable should be a written report that represents an analysis of our system and better ones might include insights that we might use and implement. What we are trying to assess here is your understanding of the open source landscape. We want to know how you evaluate libraries and I can't think of a better way to format this problem.

The deliverable should be 2-3 slides or a 2 page doc that reviews at least 2 of the following sets of libraries. (Pandas and Prefect, or the numpy+others and Streamlit, etc). You do not need to review all of them.

1. Pandas
2. Any of Numpy, Numba, Scipy, Statsmodels, CVSOPT, Dask
3. Prefect (Orchestration software)
4. Streamlit (native python dashboards)
5. Pydantic and/or FastAPI
6. A library of your choosing - needs to be one that is commonly used and has decent documentation. Choosing from the Anaconda standard library is a decent place to start. NumFocus, Apache or PyData sponsored packages are generally good too. Can be a python base library, but the bar is higher if you choose one of those (collections, itertools, etc)

Evaluation

The deliverable should show a demonstrated knowledge of these packages or strong research into them. Better responses will find strengths and weaknesses. Best answers will have code examples demonstrating those strengths and weaknesses. You can use any criteria you like to define a library's strength or weakness, some of the evaluation is related to the criteria you choose. Better answers will be specific to a function or benefit of a package rather than a generalization of the package as a whole. Better answers will compare to other libraries that have similar functionality.

Problem 5: Drinking from a fire hose

We process data quickly. The challenge is that we aren't merely warehousing the data that comes into our system, but we need to generate online, real-time analytics. Those analytics are the basis of our trading model. To do this efficiently we need to take advantage all the tools available to process data quickly. We have many models, some of them run quickly, some of them take quite a bit of time. Orchestrating those model runs is a challenge. We would like to see how you would approach the design of this type of system. You can use any language and make use of any libraries you like.

We will give you a stream of data where an object comes through the system every 2-3ms. The object will have a timestamp and a data index. The structure will look like this in decoded JSON.

```
{"timestamp": "2025-03-01 05:30:01.233901", "index": "pat000030142"}
```

You will need to run 3 models. Each model should append the timestamp for the completion of that model run.

```
{"timestamp": "2025-03-01 05:30:01.233901", "index": "pat000030142", "model1": "2025-03-01 05:30:02.528315"}
```

As you can see that timestamp suggests that model1 ran in about 1 second. Using sleeps here is how long each model should take to run.

Model 1: 3ms

Model 2: 250ms

Model 3: 1s

Not every object that comes through needs to be evaluated by every model. Since the data is coming through every 2-3 ms Model 1 should capture every other object or so if it is run linearly. Model 3 will miss quite a few.

Data

The data will be delivered via a Redis pub/sub. Your consumer of the firehose should connect to a redis pub/sub (presume it is a standard installation) and parse the messages that are sent over the pub sub.

Deliverable

Code and the "output" dataset. This output dataset is a file or database of the processed events. It should included the original timestamp and index and any timestamps appended by the models. The input dataset will run for 5 minutes.

Evaluation

The quantitative evaluation is on the timestamps. Will answer two questions. How latent were the model runs? (model timestamp minus event timestamp). How many model runs per minute for each model type - more is better. The qualitative will be around how you structure the model runs. What tools do you use to run efficiently at the various cadences. Are you able to handle all the data that comes through. What tradeoff did you make to run the models quickly as often as possible.